Chapter 1

Inventory Management System

Himanshu Gunwant (251127)
Raj Aryan Mahato (251145)

1.1 Introduction and Problem Statement

Small shops, college stores and laboratories often track their stock in notebooks or spreadsheets. This leads to incorrect quantities, no record of who moved stock and when, duplicate product entries, and no warning when an item is about to run out. Anyone who can open the spreadsheet can also change any value, so there is no access control.

The **Inventory Management System** solves this with a single, centralised C++17 backend that exposes a REST API over HTTP and stores data in an SQLite database. A lightweight browser frontend ("Stockroom") is served by the same backend, so the complete workflow runs end to end: the browser sends JSON requests, the C++ server validates them, applies the business rules inside a database transaction, and returns JSON responses.

From an end-user perspective the system lets a signed-in user:

- see a dashboard with the number of items, the total stock value and the number of low-stock items;
- browse and search items by name, SKU or category, with pagination;
- record *stock in* (goods received) and *stock out* (goods issued) with an optional note;
- view the full history of stock movements; and
- (administrators only) manage categories, suppliers, items and user accounts.

Software requirements. C++17 compiler, CMake 3.20+, the Crow HTTP framework, nlohmann/json and SQLite 3 (the last three are fetched automatically by CMake), and any modern web browser.

1.2 Scope and Feature Set

- **Authentication:** users register and log in with email and password. Passwords are stored as salted SHA-256 hashes, and the server issues an HMAC-SHA-256 signed bearer token that expires after 24 hours.
- **Role-based authorisation:** two roles, `admin` and `staff`. Admins manage master data and users; staff can view items and record stock movements.
- **Category and supplier management:** full CRUD on categories and suppliers.
- **Item management:** CRUD on items with a unique SKU, unit price, current quantity and reorder level. Supports search, category filters and pagination.
- **Transactional stock movements:** `POST /api/stock/in` and `/api/stock/out` update the quantity and write a movement record atomically. A stock-out larger than the available quantity is rejected (HTTP 409).
- **Movement history:** every change is logged with item, type, quantity, note, user and timestamp.
- **Low-stock monitoring:** lists the items whose quantity is at or below their reorder level.
- **Dashboard:** total items, total stock value, low-stock count and the five latest movements.
- **Web frontend:** a dependency-free HTML/CSS/JavaScript single-page interface, served by the backend at `http://localhost:8080`.

Table 1.1: REST API endpoints

Method	Endpoint	Access
GET	/api/health	Public
POST	/api/auth/register	First user / Admin
POST	/api/auth/login	Public
GET	/api/auth/me	Authenticated
GET, POST	/api/categories	Auth / Admin
PUT, DELETE	/api/categories/{id}	Admin
GET, POST	/api/suppliers	Auth / Admin
PUT, DELETE	/api/suppliers/{id}	Admin
GET, POST	/api/items	Auth / Admin
GET, PUT, DELETE	/api/items/{id}	Auth / Admin
GET	/api/items/low-stock	Authenticated
POST	/api/stock/in, /api/stock/out	Authenticated
GET	/api/stock/movements	Authenticated
GET	/api/dashboard	Authenticated

1.3 System Architecture

The backend follows a layered architecture (Figure 1.1). HTTP handling, business logic and database access are kept in separate modules, so each layer depends only on the one below it.

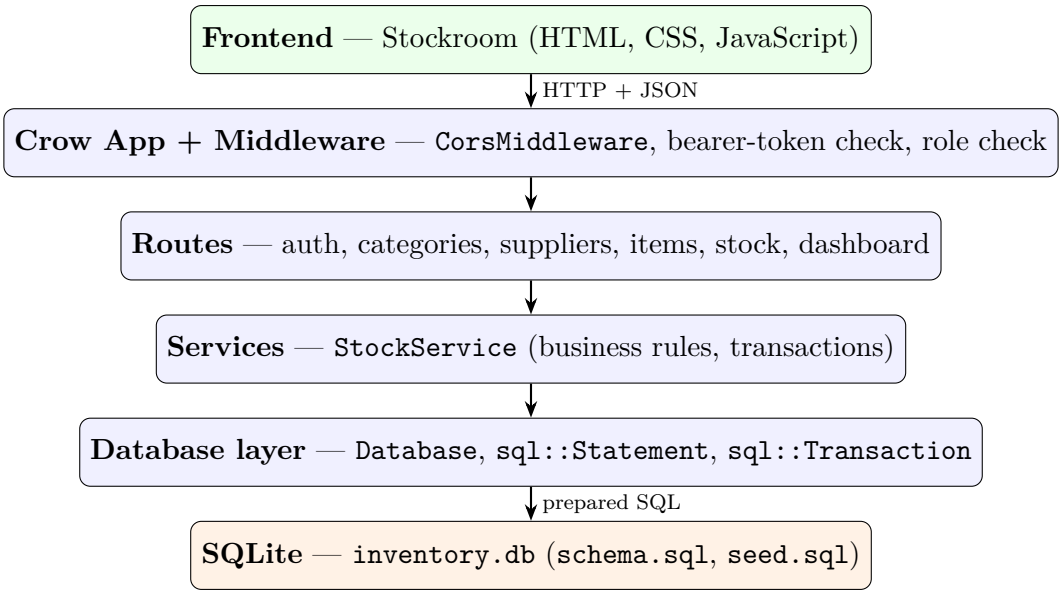


Figure 1.1: Layered architecture of the Inventory Management System

Database design. The relational schema has five tables: `users`, `categories`, `suppliers`, `items` and `stock_movements`. `items` references `categories` and `suppliers` (`ON DELETE SET NULL`), and `stock_movements` references `items` (`ON DELETE CASCADE`) and `users`. `CHECK` constraints enforce non-negative prices and quantities, positive movement quantities, and valid roles and movement types.

1.4 Object-Oriented Architecture

The project uses C++ classes to model resources, hide implementation details and manage resources safely. The OOP principles applied are:

- **Encapsulation:** `TokenStore` keeps its token map, HMAC secret and mutex `private`, and exposes only `issue()` and `find()`. `Database` hides the raw `sqlite3*` handle behind a `std::unique_ptr`.
- **Abstraction:** route handlers call `StockService::move()` without knowing the SQL or locking logic behind it. The `sql` helpers (`prepare`, `bindText`, `bindInt`) hide the low-level SQLite C API.
- **RAII (constructors and destructors):** `Database` opens the connection in its constructor and a custom deleter closes it. `sql::Transaction` begins a transaction in its constructor and automatically rolls back in its destructor unless `commit()` was called. Every `sqlite3_stmt` is wrapped in a `unique_ptr` with `StatementDeleter`.

- **Polymorphism:** `response::success()` is *overloaded* for `nlohmann::json` and `crow::json::wvalue`. Compile-time (static) polymorphism is used through the class template `crow::App<CorsMiddleware>`. Crow calls the middleware's `before_handle()` and `after_handle()` hooks without any virtual base class.
- **Composition and aggregation:** see Section 1.4.1.
- **Copy control:** `sql::Transaction` deletes its copy constructor and copy assignment operator, so a transaction can never be committed or rolled back twice.
- **Exception handling:** constructors throw `std::runtime_error` on failure, and handlers catch exceptions and convert them into JSON error responses.

1.4.1 Class Design and Associations

- `Database` — owns one SQLite connection and a `std::mutex` that serialises access from Crow's worker threads. *Composition:* the connection handle and the mutex are created and destroyed together with the `Database` object.
- `sql::Transaction` — RAII wrapper for `BEGIN IMMEDIATE` / `COMMIT` / `ROLLBACK`. It uses (depends on) a `sqlite3*` but does not own it.
- `StockService` — the business-logic class for stock in/out, movement history and low stock. It holds a *reference* to a `Database` that is created in `main()` and outlives it, which is *aggregation*. It returns a `StockResult` value object (`success`, error `code`, new `quantity`).
- `TokenStore` — issues and validates signed bearer tokens. It is *composed of* a private `unordered_map<string, Entry>`, where each nested `Entry` contains a `Principal` and an expiry time.
- `Principal` — the authenticated user (`id`, `name`, `email`, `role`).
- `CorsMiddleware` — a Crow middleware type that adds CORS headers to every response. It is passed as a template argument to `crow::App`.
- `ItemInput` — a data-transfer object filled by `parseItem()` after validating the JSON body.

Inheritance was deliberately not used. The domain has no natural "is-a" hierarchy, so the design prefers composition, aggregation and templates, which keep the classes small and avoid virtual-call overhead.

1.5 UML Diagrams

1.5.1 Use Case Diagram

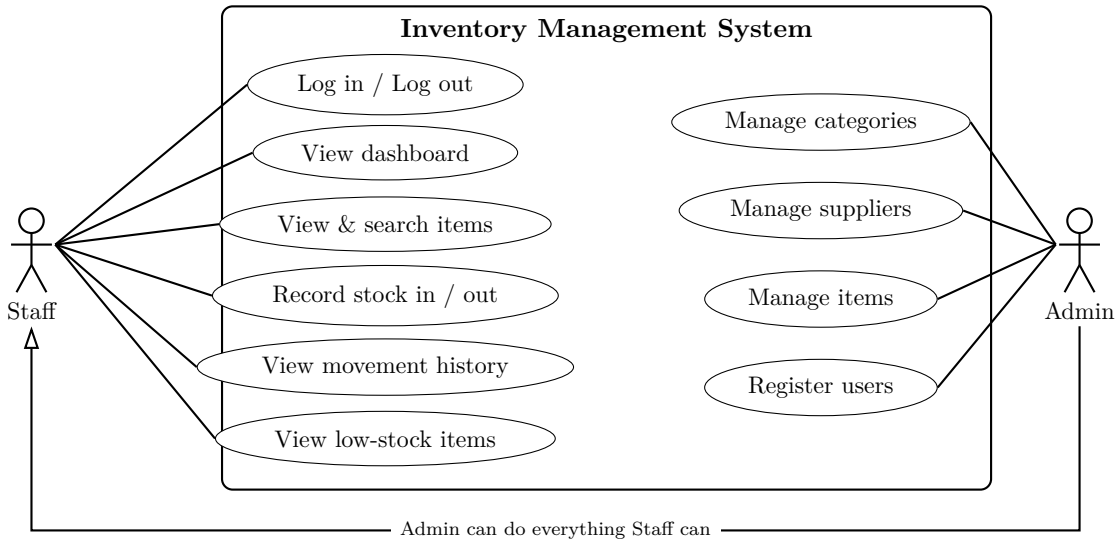


Figure 1.2: Use case diagram

1.5.2 Class Diagram

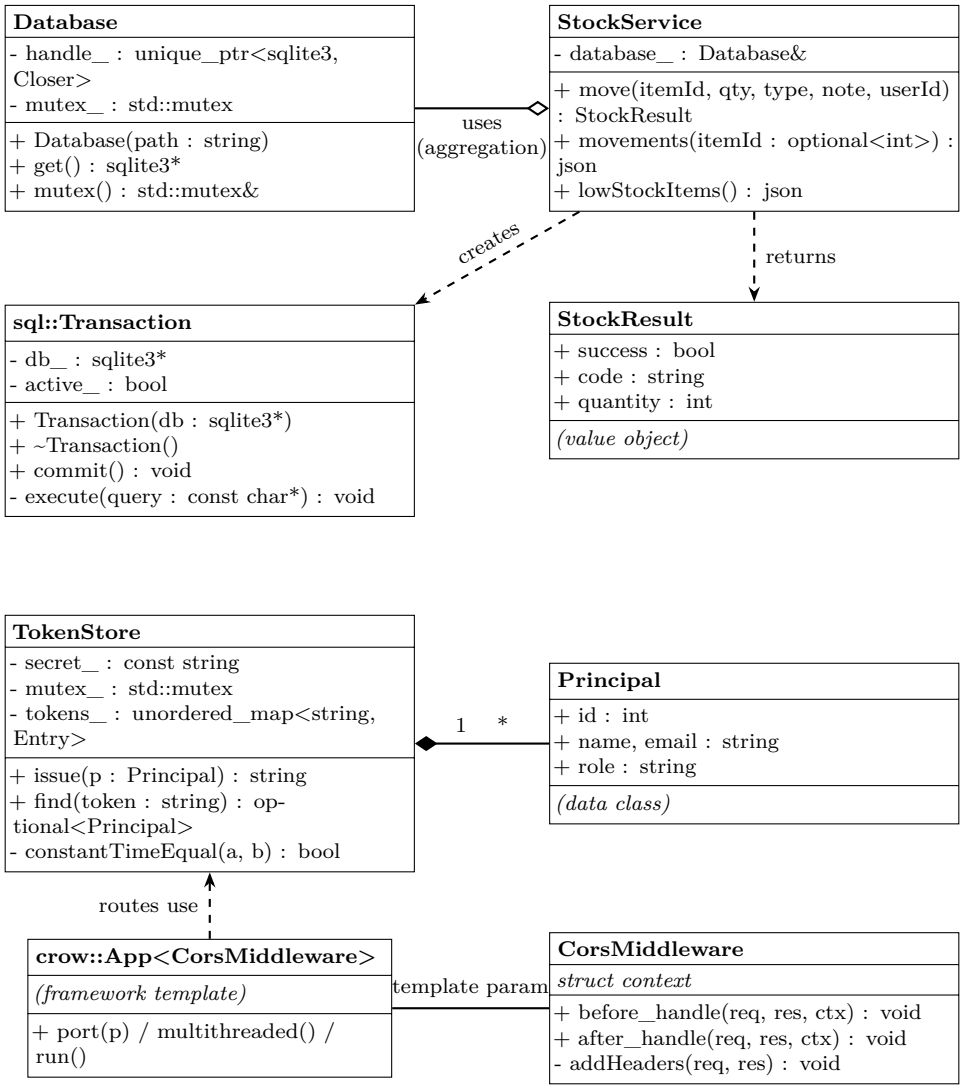


Figure 1.3: Class diagram of the main backend classes

1.5.3 Sequence Diagram: Stock Out

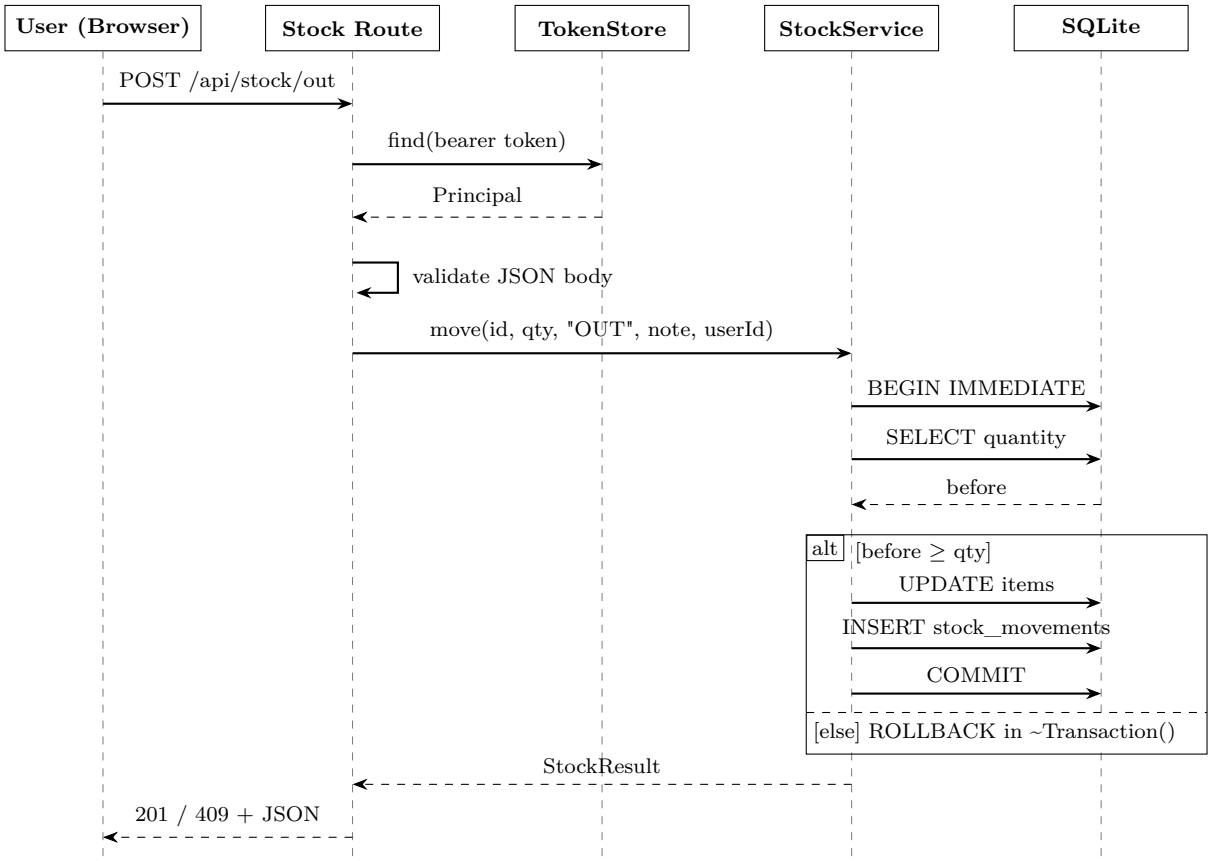


Figure 1.4: Sequence diagram for recording a stock-out

1.5.4 Activity Diagram: Recording a Stock Movement

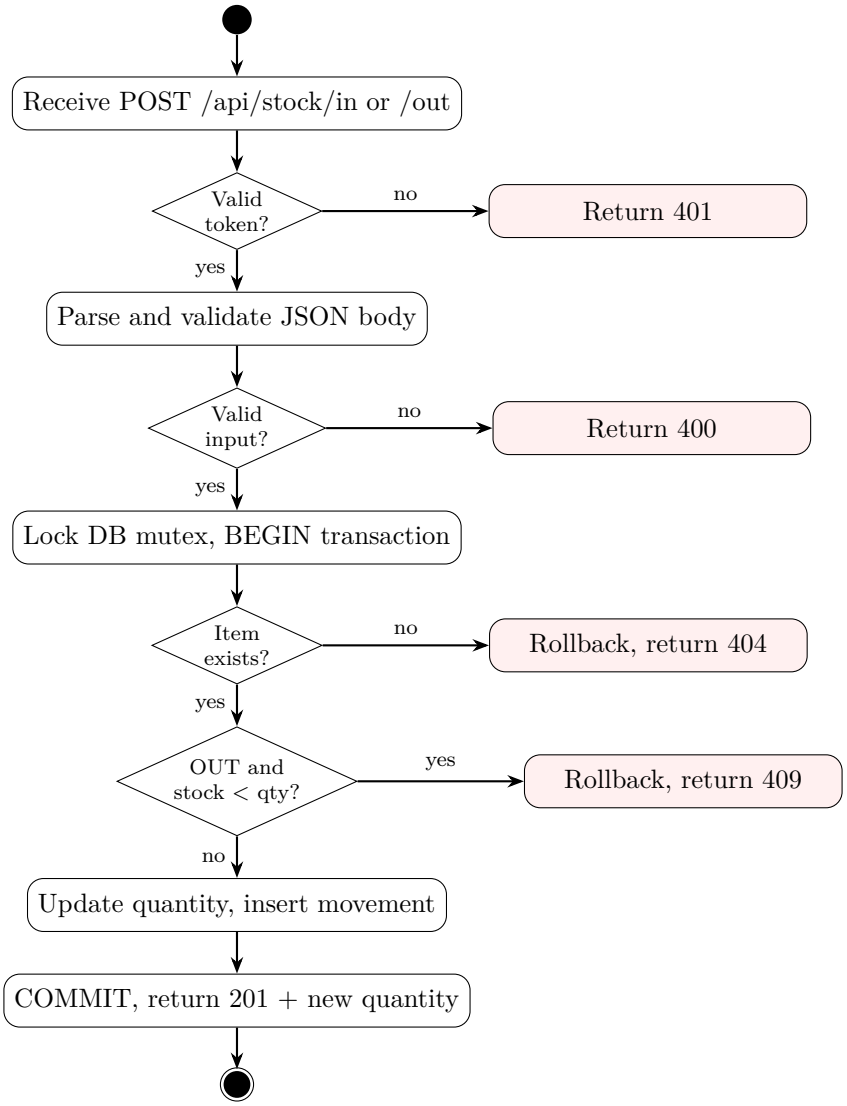


Figure 1.5: Activity diagram for the stock in/out workflow

1.6 Implementation: Key Code Snippets

1.6.1 RAII Database Wrapper (Encapsulation and Composition)

```
class Database {
  public:
    explicit Database(const std::string &path);
    sqlite3 *get() const noexcept { return handle_.get(); }
    std::mutex &mutex() noexcept { return mutex_; }

  private:
    struct Closer {
        void operator()(sqlite3 *db) const noexcept { if (db)
           sqlite3_close(db); }
    };
```

```
11      std::unique_ptr<sqlite3, Closer> handle_;    // closed
            automatically
12      std::mutex mutex_;
13  };
```

Listing 1.1: `Database` class: owns the SQLite connection and its mutex

1.6.2 Transaction Class (RAII and Copy Control)

```
1  class Transaction {
2    public:
3      explicit Transaction(sqlite3 *db) : db_(db) { execute("BEGIN
           IMMEDIATE"); }
4      Transaction(const Transaction &) = delete;              //
           non-copyable
5      Transaction &operator=(const Transaction &) = delete;
6      ~Transaction() {
7          if (active_) sqlite3_exec(db_, "ROLLBACK", nullptr, nullptr,
               nullptr);
8      }
9      void commit() { execute("COMMIT"); active_ = false; }
10
11   private:
12     void execute(const char *query);   // throws std::runtime_error
           on failure
13     sqlite3 *db_;
14     bool active_ = true;
15 };
```

Listing 1.2: `sql::Transaction`: rollback in the destructor unless committed

1.6.3 Business Logic in the Service Layer (Abstraction)

```
1  StockResult StockService::move(int itemId, int amount, const
       std::string &type,
2                                 const std::string &note, int userId) {
3      if (itemId <= 0 || amount <= 0 || userId <= 0 || (type != "IN"
           && type != "OUT"))
4          return {false, "INVALID_INPUT", 0};
5      try {
6          std::lock_guard<std::mutex> lock(database_.mutex());
7          sql::Transaction transaction(database_.get());
8          auto lookup = sql::prepare(database_.get(),
9                                     "SELECT quantity FROM items WHERE
                                         id=?");
10         sql::bindInt(lookup.get(), 1, itemId);
11         if (sqlite3_step(lookup.get()) != SQLITE_ROW) return {false,
               "ITEM_NOT_FOUND", 0};
12         const int before = sqlite3_column_int(lookup.get(), 0);
13         if (type == "OUT" && before < amount) return {false,
               "INSUFFICIENT_STOCK", before};
14         // ... UPDATE items ..., then INSERT INTO
               stock_movements(...) ...
15         transaction.commit();                          // otherwise
               destructor rolls back
```

```
        return {true, "", type == "IN" ? before + amount : before -
            amount};
    } catch (const std::exception &) {
        return {false, "DATABASE_ERROR", 0};
    }
}
```

Listing 1.3: `StockService::move()` (abridged): atomic stock update and movement log

1.6.4 Token Store (Encapsulation)

```
struct Principal { int id{}; std::string name; std::string email;
    std::string role; };

class TokenStore {
  public:
    std::string issue(Principal principal);                    //
        nonce + "." + HMAC
    std::optional<Principal> find(const std::string &token); //
        verify + expiry check
  private:
    static bool constantTimeEqual(const std::string &l, const
        std::string &r);
    struct Entry { Principal principal;
        std::chrono::system_clock::time_point expires; };
    const std::string secret_ = password::randomSalt();
    std::mutex mutex_;
    std::unordered_map<std::string, Entry> tokens_;
};
```

Listing 1.4: `TokenStore`: private state behind a two-method interface (abridged)

1.6.5 Polymorphism: Function Overloading and Template Middleware

```
namespace inventory::response {
inline crow::response success(const nlohmann::json &data, int status
    = 200);
inline crow::response success(crow::json::wvalue data, int status =
    200);
inline crow::response error(int status, const std::string &message);
}

struct CorsMiddleware {
    struct context {};
    void before_handle(crow::request &req, crow::response &res,
        context &);
    void after_handle(crow::request &req, crow::response &res,
        context &);
};

crow::App<CorsMiddleware> app;    // static polymorphism via templates
inventory::routes::registerStockRoutes(app, stockService, tokens);
app.port(8080).multithreaded().run();
```

Listing 1.5: Overloaded `response::success()` and the middleware passed as a template argument

1.7 Project Screenshots

The screenshots below were taken from the Stockroom frontend running against the C++ backend at `http://localhost:8080`. The "API connected" indicator in the sidebar confirms that the browser is talking to the REST API.

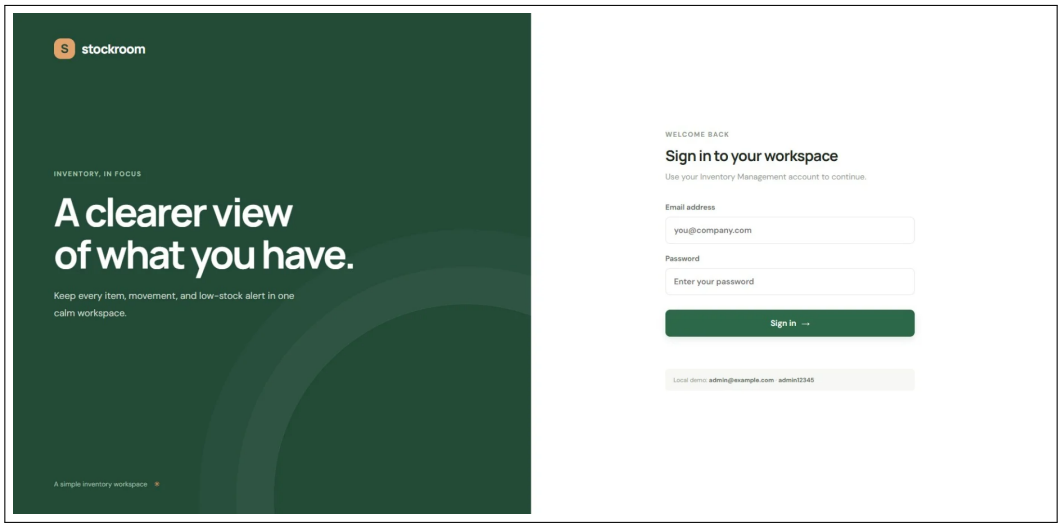


Figure 1.6: Sign-in page: credentials are sent to `POST /api/auth/login`, which returns a bearer token

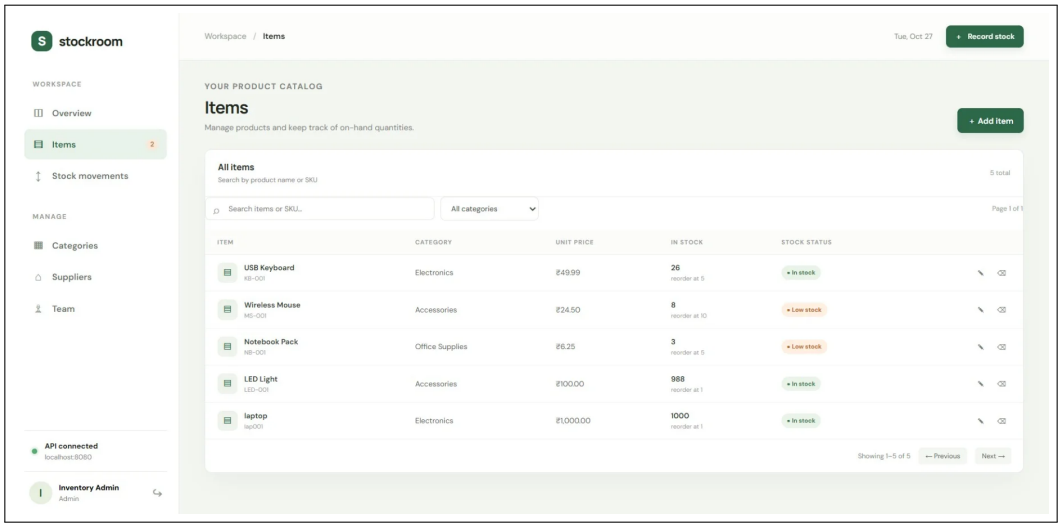


Figure 1.7: Items catalogue with search, category filter, pagination and low-stock status (`GET /api/items`)

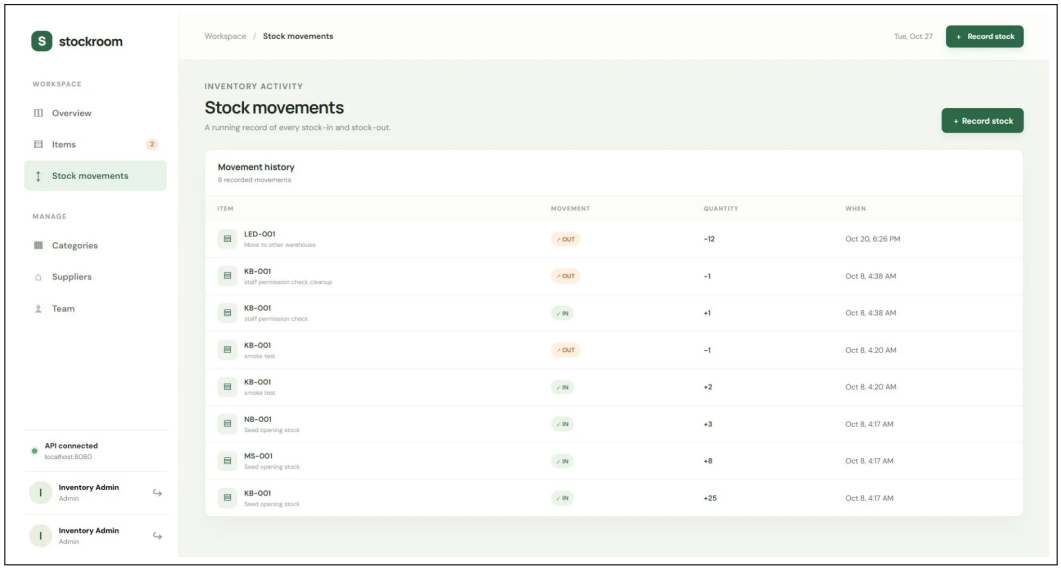


Figure 1.8: Stock movement history recorded by `StockService` (`GET /api/stock/movements`)

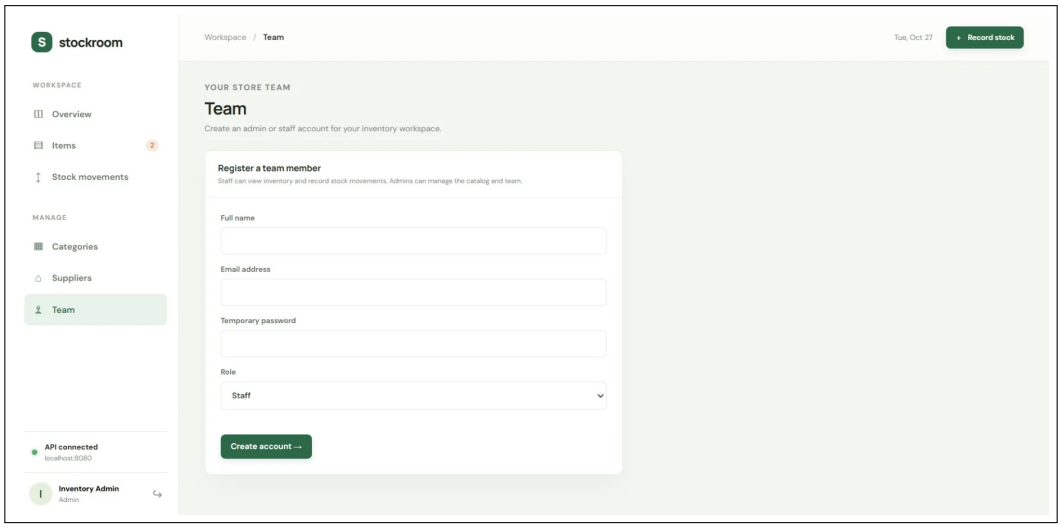


Figure 1.9: Team page: an admin registers a staff or admin account (`POST /api/auth/register`)

1.8 Conclusion and Future Work

The Inventory Management System integrates a browser frontend, a C++17 REST API and an SQLite database into one end-to-end workflow. Classes such as `Database`, `sql::Transaction`, `StockService` and `TokenStore` show encapsulation, abstraction, RAII, composition, aggregation and polymorphism in practice. Transactional stock updates keep quantities and the movement history consistent.

Future work includes reading secrets (such as the token signing key) from environment variables with `std::getenv`, using stronger password hashing (bcrypt or Argon2), persistent sessions, support for multiple warehouses, purchase and sales orders, CSV/PDF reports, and automated tests.